

Maritime CargoService: Sprint 0

Davide Chirichella, Gabriele Doti, Daniele
Maccagnan

Ingegneria dei Sistemi Software, A.A. 2025/2026
Alma Mater Studiorum . Università di Bologna

June 23, 2026

1. Introduction

Una compagnia di trasporto marittimo di container (d'ora in poi *la committente*) intende automatizzare le operazioni di carico dei container nella hold della nave (d'ora in poi **hold**). A tal fine prevede di impiegare un robot a guida differenziale (Differential Drive Robot, d'ora in poi **cargorobot**).

L'obiettivo dello Sprint 0 è formalizzare i requisiti forniti dalla committente in modo preciso e non ambiguo, costruire un primo modello logico dei macro-componenti del sistema, evidenziare il *core business*, motivare la scelta del linguaggio di modellazione e definire un primo insieme di piani di test funzionali.

Ogni scelta è strettamente ancorata ai requisiti: il modello qui presentato serve a catturare il comportamento richiesto, senza anticipare decisioni progettuali o scelte implementative che verranno affrontate negli sprint successivi.

La traccia del progetto può essere scaricata dal seguente [link](#)^o

2. Requirements

L'azienda richiede di realizzare un servizio denominato **cargoservice** con il seguente funzionamento:

- Il cargoservice è in grado di ricevere una richiesta di carico di un container inviata da un cliente tramite il pulsante dell'IOPort.
- Invia la risposta **retrylater** se l'IOPort è attualmente occupato da un container oppure se il sistema è **Out of service**.
- Rifiuta la richiesta quando la hold è già piena, ovvero gli slot1-4 sono già tutti occupati.
- Altrimenti, considera il sistema come **engaged**, rileva uno slot libero e restituisce come risposta il nome dello slot riservato. Mentre il sistema è engaged, il LED deve lampeggiare.
- Quando la richiesta di carico viene accettata, il cliente deve spostare il container nell'area del sensore entro un tempo prefissato (ad esempio 30 secondi), altrimenti il sistema diventa **disengaged**.
- Successivamente, il cargoservice utilizza il cargorobot per spostare il container dall'IOPort a slot5 (per l'etichettatura del container) e poi allo slot riservato.
- Il servizio deve inoltre mostrare sul display dell'IOPort:
 - lo stato attuale della hold
 - il messaggio "**Service working**" quando tutto sta procedendo correttamente
 - il messaggio "**Out of service**" se il sensore sonar misura una distanza $D > D_{FREE}$ per almeno 3 secondi (possibile guasto del sonar)

2.1. Domande aperte alla committente

Domanda al committente.

Posizione dell'IOPort nella mappa. I requisiti indicano che il cargorobot sposta

il container dall'IOPort a slot5, lasciando intendere che l'IOPort sia una cella praticabile. Come si colloca nella griglia?

Risposta della committente La collocazione dell'IOPort nella griglia è da intendersi informalmente come mostrato nella figura presente nella traccia.

Domanda al committente.

Richieste concorrenti. Il sistema deve bufferizzare più richieste di carico contemporanee, oppure una nuova richiesta ricevuta mentre il sistema è engaged viene semplicemente refusata / respinta con retrylater senza accodamento? Dai requisiti si interpreta che le richieste **non siano bufferizzate**: se il sistema è occupato, la risposta è immediata (retrylater o refused) senza code di attesa.

Risposta della committente Si conferma che le richieste non devono essere bufferizzate. Il sistema accetta le richieste mediante l'IOPort e, se questa risulta occupata, non deve essere possibile inviare altre richieste.

Domanda al committente.

Liberazione degli slot e aggiornamento della disponibilità. I requisiti descrivono il processo di carico dei container negli slot1-4, ma non specificano come venga gestita la liberazione di uno slot già occupato. Possiamo assumere che lo svuotamento degli slot sia effettuato da sistemi o operatori esterni al nostro sistema? In tal caso, con quale meccanismo viene notificato a cargoservice che uno specifico slot è stato liberato e può tornare disponibile per future prenotazioni?

Risposta della committente Non è previsto lo svuotamento degli slot. Sarà l'obiettivo di un progetto futuro.

Domanda al committente.

Ripristino dello stato Service working. I requisiti specificano che il sistema entra nello stato Out of service quando il sonar rileva una distanza $D > D_{FREE}$ per almeno 3 secondi. Non è invece specificata la condizione per il ritorno allo stato Service working. Il sistema deve tornare operativo non appena il sonar rileva $D \leq D_{FREE}$ oppure è richiesto un ulteriore intervallo di stabilità prima del ripristino del servizio?

Risposta della committente Si conferma che l'interpretazione è corretta.

3. Requirement analysis

3.1. Core business

Il **core business** del sistema è la gestione del ciclo di carico di un container. La responsabilità della sequenza applicativa resta in **cargoservice**: il cargorobot è coinvolto per eseguire spostamenti richiesti dal servizio, non per decidere se una richiesta debba

essere accettata, rifiutata o sospesa. La sequenza principale ricavata dai requisiti è espressa dal metamodello ccome segue:

```
System cargosystem

// Vocabolario delle interazioni
Request load_request : loadRequest(none)
Reply load_accepted  : loadAccepted(slotID)    for load_request
Reply load_retrylater : loadRetryLater(none)    for load_request
Reply load_refused    : loadRefused(none)       for load_request

Event sonar_distance : distance(D)
Dispatch marking_done : markingDone(containerID)

Context ctxcargoservice ip [host="localhost" port=8050]

// Core Business
QActor cargoservice context ctxcargoservice {

    State s0 initial {
        println("cargoservice | started")
    }
    Goto disengaged

    State disengaged {
        println("cargoservice | DISENGAGED: waiting for load_request...")
    }
    Transition t0
        whenRequest load_request → handle_load_request

    State handle_load_request {
        printCurrentMessage
        // MOCK Sprint 0: Simuliamo l'accettazione (Caso Felice)
        // La logica reale di calcolo slot e OoS sarà nello Sprint 1
        replyTo load_request with load_accepted : loadAccepted(1)
    }
    Goto engaged

    State engaged {
        println("cargoservice | ENGAGED: waiting for container (sonar)...")
    }
    Transition t1
        whenTime 10000 → handle_timeout          // Requisito: timeout
deposito
        whenEvent sonar_distance → move_to_slot5 // Requisito: rilevamento
container

    State handle_timeout {
        println("cargoservice | TIMEOUT: container non depositato, libero
l'IOPort")
    }
    Goto disengaged

    State move_to_slot5 {
        println("cargoservice | CONTAINER RILEVATO: avvio movimentazione
```

```

    verso slot 5")
        // Qui ci sarà la delega al cargorobot (Sprint futuri)
    }
    Goto disengaged
}

```

3.2. Macro-componenti e natura software

COM- PO- NENTE	RUOLO	STATO DI DISPONI- BILITÀ	NATURA SOFTWARE
car-goser-vice	Orchestratore centrale del sistema. Riceve le richieste di carico, verifica le pre-condizioni della hold, valida i timeout e coordina cargorobot e marker.	Da sviluppare	Si ritiene opportuno rappresentare cargoservice come un QAK Actor, dato che deve gestire il ciclo di business, reagire in tempo reale agli eventi del sonar e inviare comandi.
car-gorobot	Entità logica che si occupa del movimento del DDR. Riceve i comandi di trasferimento e sposta i container dall'IOPort allo slot5, e successivamente allo slot finale riservato.	Da sviluppare (Mock in Sprint 0)	Si ritiene opportuno rappresentare il cargorobot come un QAK Actor: dato che deve governare i movimenti del DDR all'interno della hold, necessita di un controllo autonomo per poter ricevere ed eseguire in modo asincrono le richieste di movimento.
IOPort	Interfaccia virtuale di Input/Output, ha pushbutton e display. Il primo rileva la richiesta di carico, mentre il secondo mostra i messaggi di stato del servizio e della hold.	Da sviluppare (Mock in Sprint 0)	Si ritiene opportuno rappresentare IOPort come un QAK Actor. Le funzionalità del pushbutton richiedono un funzionamento proattivo (per generare le request), e il display un funzionamento reattivo per mostrare i messaggi di stato.
sensore	Sorgente logica di eventi. Simula le misurazioni sulla distanza di un eventuale container dalla	Da sviluppare (Mock in Sprint 0)	Si ritiene opportuno rappresentare il sensore come QAK Actor, essendo un'entità attiva che effettua misurazioni ed emette eventi in maniera completamente autonoma.

	sensor_area per comunicarle al sistema.		
marker device	Dispositivo che fornisce un codice a barre ai container nello slot5, segnalando poi l'avvenuto completamento.	Da sviluppare (Mock in Sprint 0)	Si ritiene opportuno rappresentare il marker come un QAK Actor, in grado di ricevere comandi ed emettere conferme.
LED	Indicatore lampeggiante di stato del sistema (engaged o disengaged).	Da sviluppare (Mock in Sprint 0)	Si ritiene opportuno rappresentare il LED come un QAK Actor (attuatore reattivo). Solo in veste di attore dotato di message queue autonoma può ricevere ed elaborare correttamente i messaggi di Dispatch asincroni inviati dal cargoservice.
hold	Rappresentazione e gestione dello stato della hold, tiene traccia degli slot occupati.	Da sviluppare	Si ritiene opportuno rappresentare la hold come QAK Actor (oppure come POJO incapsulato in cargoservice), permettendo così di separare la logica dei dati dalla logica di business.

3.3. Formalizzazione dei messaggi QAK

La seguente formalizzazione non introduce ancora scelte di progetto, limitandosi ad elencare i messaggi necessari per esprimere i requisiti. **Request/Reply** viene usato quando il requisito prevede una risposta osservabile. ; **Event** viene invece tipicamente usato quando l'emettitore non sa a chi arriverà il messaggio e si “preoccupa” solamente di emettere informazioni.

3.3.1. customer / IOPort -> cargoservice

```
Request  load_request    : loadRequest(none)
Reply    load_accepted   : loadAccepted(slotID)    for load_request
Reply    load_retrylater : loadRetryLater(none)    for load_request
Reply    load_refused    : loadRefused(none)       for load_request
```

La Request `load_request` rappresenta il momento nel quale viene effettuata una richiesta tramite il pushbutton dell'IOPort e le 3 Reply sono le relative risposte del cargoservice: nel caso la richiesta venisse accettata si consegna anche l'ID dello slot nel quale va posizionato il carico.

3.3.2. Sonar ->

```
Event sonar_distance : distance(0)
```

Questo evento rappresenta la comunicazione dei propri dati da parte del sonar al sistema. Probabilmente sarà cargoservice ad “ascoltare” questo evento.

3.3.3. marker device -> cargoservice

```
Dispatch marking_done : markingDone(containerID)
```

Messaggio di conferma a cargoservice di fine lavoro del marker.

3.3.4. cargoservice -> display / LED

```
Dispatch display_hold_state : holdState(state)
Dispatch display_status      : displayStatus(message)
Dispatch led_on              : ledOn(none)
Dispatch led_off             : ledOff(none)
```

Questi Dispatch rappresentano rispettivamente la “consegna” dei dati sullo status generale e sullo stato della hold da parte del cargoservice al display e le richieste di accendere o spegnere il LED. Le richieste al LED avverranno in corrispondenza di eventuali cambi di stato (**engaged** o **disengaged**).

3.4. Contesti logici

CONTESTO	COMPONENTI E RESPONSABILITÀ
ctxCargoService	Contiene l'attore cargoservice. Nucleo del comportamento richiesto e punto di orchestrazione del ciclo di carico.
ctxCustomer	Raggruppa le entità dedicate all'interazione con l'utente: IO-Port (con display e pushbutton) e LED.
ctxDevices	Raggruppa i dispositivi presenti nella hold: sonar, hold e markerdevice.
ctxRobot	Raggruppa le entità legate al cargorobot e alla movimentazione richiesta.

3.5. Schema della hold

Legenda: H = HOME S = sonar/sensor area 1-4 = slot1-4
5 = slot5 I = IOPort X = ostacolo . = libero

```
col0 col1 col2 col3 col4 col5 col6
riga0 [ H ][ S ][ . ][ . ][ . ][ . ][ . ]
riga1 [ . ][ 1 ][ X ][ X ][ 2 ][ . ][ . ]
riga2 [ . ][ . ][ . ][ . ][ X ][ 5 ][ . ]
```

```

riga3 [ . ][ 3 ][ X ][ X ][ 4 ][ . ][ . ]
riga4 [ . ][ . ][ . ][ . ][ . ][ . ][ . ]
riga5 [ I ][ X ][ X ][ X ][ X ][ X ][ X ]

```

Nota: La posizione esatta dell'IOPort e del sonar richiede conferma dalla committente. Lo schema è una prima interpretazione fedele alla figura allegata ai requisiti.

4. Test plan

I test funzionali verificano il comportamento osservabile del sistema rispetto ai requisiti, indipendentemente dall'implementazione.

SCENARIO	PRECONDIZIONI	RISULTATO ATTESO
Accettazione	disengaged , non OoS, IOPort libera, slot disponibile	load_accepted con slot riservato; stato engaged ; LED lampeggiante
Rifiuto (hold piena)	disengaged , non OoS, IOPort libera, slot1–4 tutti occupati	load_refused ; stato disengaged ; LED spento
Retrylater (OoS)	Sistema Out of service	load_retrylater ; stato immutato
Retrylater (IOPort occupata)	disengaged , non OoS, IOPort occupata da un container	load_retrylater ; stato disengaged
Timeout deposito	Sistema engaged ; customer non deposita entro il tempo prefissato	Sistema disengaged ; LED spento
Ciclo completo	disengaged , non OoS, IOPort libera, slot disponibile	Container nello slot riservato; display “ Service working ”; disengaged ; LED spento
Guasto sonar	Sistema operativo; sonar misura $D > D_{FREE}$ per ≥ 3 s	Sistema Out of service ; display “ Out of service ”
Rilevazione container	Sistema engaged ; sensor area vuota; sonar misura $D < D_{FREE}/2$ per ≥ 3 s	cargoservice avvia la movimentazione verso slot5

5. Project

Dall'analisi dei requisiti si propone, come ipotesi iniziale, una struttura a **tre sprint** con integrazione progressiva dei componenti. La suddivisione serve a organizzare il lavoro e i test, non a fissare decisioni architetturali definitive.

5.1. Sprint 1: Core business

Goal: realizzare il primo nucleo eseguibile di cargoservice con collaboratori simulati.

Funzionalità: ciclo di carico completo con collaboratori simulati, modello dello stato della hold, LED simulato, timeout.

5.2. Sprint 2: IOPort, display e sonar

Goal: rendere esplicita l'interazione con IOPort, display e sonar a livello software.

Funzionalità: IOPort come interfaccia web, sonar simulato (rilevazione container e malfunzionamento), aggiornamento display con stato e messaggi.

5.3. Sprint 3: Dispositivi fisici

Goal: integrare, dove disponibili, i dispositivi fisici o i servizi forniti e verificare il comportamento end-to-end.

Funzionalità: cargorobot e servizio DDR disponibile, sonar fisico, marker device fisico, LED fisico.

6. Team di lavoro

NOME E COGNOME	MATRICOLA
Davide Chirichella	0001222371
Gabriele Doti	0001245897
Daniele Maccagnan	0001247340